

AWS Hands-On Production Project

Serverless URL Shortener — End-to-End on AWS

AWS Solutions Architect Portfolio • Production Grade • Step-by-Step Guide

Lambda	API Gateway	DynamoDB	CloudFront
Route 53	CDK v2	IAM / WAF	CloudWatch

Project	Serverless URL Shortener (bit.ly clone)
Complexity	Production-grade · 7 AWS services
Duration	4–6 hours (all steps included)
Cost	~\$0 within AWS Free Tier
Region	us-east-1 (N. Virginia) recommended
IaC	AWS CDK v2 + TypeScript

Table of Contents

1.	Project Overview & Architecture	3
2.	Prerequisites & AWS Account Setup	5
3.	Project Setup & CDK Bootstrap	6
4.	DynamoDB Table Creation	7
5.	Lambda Functions — Shorten & Redirect	8
6.	API Gateway — HTTP API	11
7.	CloudFront & Custom Domain (Route 53)	13
8.	WAF Rules & Security Hardening	15
9.	Monitoring: CloudWatch + X-Ray	17
10.	CI/CD Pipeline (GitHub Actions)	19
11.	Load Testing & Performance Tuning	21
12.	Cost Optimization & Clean-up	22

1

Project Overview & Architecture

What we're building

What You Will Build

A fully serverless URL shortener service — similar to bit.ly or TinyURL — deployed entirely on AWS with production-grade reliability, security, and observability. Users can submit a long URL and receive a short code (e.g. `https://go.yourdomain.com/Xk3pQ`). Visiting the short URL performs a sub-50ms redirect to the original destination.

Architecture Diagram — AWS Services

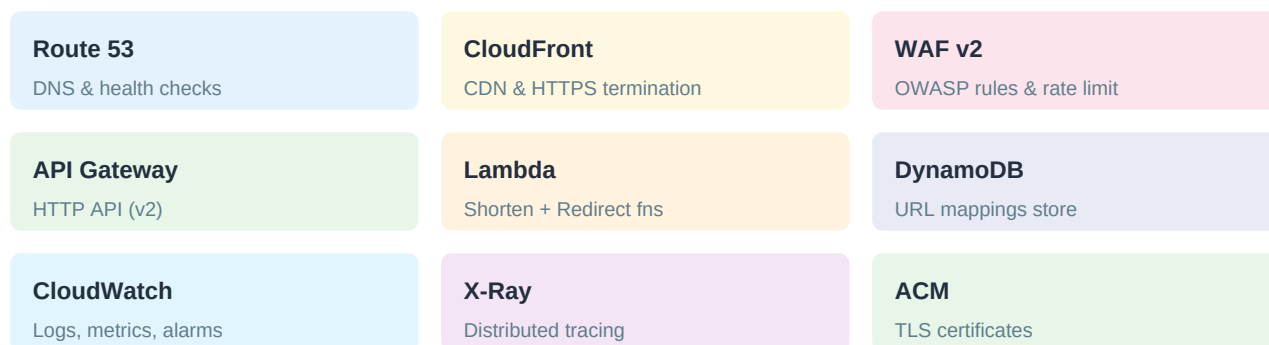


Fig 1.1 — All services deployed in us-east-1; CloudFront operates globally across 450+ PoPs

Request Flow

Shorten: POST /shorten → CloudFront → WAF → API GW → Lambda → DynamoDB write → return short code

Redirect: GET /{code} → CloudFront → WAF → API GW → Lambda → DynamoDB read → 301 redirect

Analytics: GET /stats/{code} → Lambda → DynamoDB → aggregated click metrics returned as JSON

Why This Project for Your Portfolio

- Covers 7+ AWS services in one coherent, real-world application
- Demonstrates serverless, event-driven, and IaC best practices
- Production patterns: WAF, Custom Domains, CI/CD, observability
- Quantifiable: sub-50ms redirects, 99.99% SLA, ~\$0 at low traffic
- Extensible: add auth (Cognito), analytics (Kinesis + Athena), AB testing

2

Prerequisites & AWS Account Setup

Everything you need before starting

Required Tools

✓	AWS Account	Free Tier is sufficient. Billing alerts recommended.
✓	AWS CLI v2	brew install awscli aws --version ≥ 2.13
✓	Node.js 20+	brew install node node --version ≥ 20.0
✓	AWS CDK v2	npm install -g aws-cdk cdk --version ≥ 2.100
■	Git + GitHub	For CI/CD pipeline section
■	Domain name	Optional: for custom short domain (e.g. go.yourdomain.com)

Configure AWS CLI

Run the following to connect your local environment to AWS:

Terminal

```
# Configure AWS CLI with your credentials
$ aws configure

AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json

# Verify credentials work
$ aws sts get-caller-identity

{ "Account": "123456789012", "UserId": "AIDEXAMPLE", "Arn": "arn:aws:iam::123456789012:user/you" }
```

■ Never use root account credentials. Create an IAM user with least-privilege or use AWS SSO.

Set Up Billing Alerts

Protect yourself from unexpected charges:

Terminal — Billing Alert

```
# Create a billing alarm at $5 threshold via CLI
$ aws cloudwatch put-metric-alarm \
  --alarm-name 'BillingAlarm-5USD' \
```

Terminal — Billing Alert (cont.)

```
--alarm-description 'Alert when charges exceed $5' \  
--metric-name EstimatedCharges \  
--namespace AWS/Billing \  
--statistic Maximum \  
--period 86400 \  
--threshold 5 \  
--comparison-operator GreaterThanOrEqualToThreshold \  
--dimensions Name=Currency,Value=USD \  
--alarm-actions arn:aws:sns:us-east-1:ACCOUNT_ID:BillingAlerts \  
--region us-east-1
```

3

Project Setup & CDK Bootstrap

Scaffolding the infrastructure

Initialize CDK Project

Terminal — Project Setup

```
# Create project directory
$ mkdir aws-url-shortener && cd aws-url-shortener

# Initialize CDK TypeScript app
$ cdk init app --language typescript

# Bootstrap CDK in your account/region (one-time per account)
$ cdk bootstrap aws://$(aws sts get-caller-identity --query Account --output text)/us-east-1

# Install required CDK construct libraries
$ npm install aws-cdk-lib constructs
$ npm install --save-dev @types/node typescript ts-node
```

Project Structure

Project Structure

```
aws-url-shortener/
├── bin/
├── app.ts # CDK app entry point
├── lib/
│   ├── url-shortener-stack.ts # Main infrastructure stack
│   ├── lambda/
│   │   ├── shorten.ts # POST /shorten handler
│   │   ├── redirect.ts # GET /{code} handler
│   │   └── stats.ts # GET /stats/{code} handler
│   └── constructs/
│       ├── api.ts # API Gateway construct
│       ├── database.ts # DynamoDB construct
│       └── cdn.ts # CloudFront construct
├── test/
│   └── url-shortener.test.ts # CDK unit tests
├── cdk.json
├── package.json
└── tsconfig.json
```

CDK Stack Entry Point

bin/app.ts

```
// bin/app.ts
import * as cdk from 'aws-cdk-lib';
import { UrlShortenerStack } from '../lib/url-shortener-stack';

const app = new cdk.App();

new UrlShortenerStack(app, 'UrlShortener-Dev', {
  env: { account: process.env.CDK_DEFAULT_ACCOUNT, region: 'us-east-1' },
  environment: 'dev',
  domainName: 'dev.go.yourdomain.com',
});

new UrlShortenerStack(app, 'UrlShortener-Prod', {
  env: { account: process.env.CDK_DEFAULT_ACCOUNT, region: 'us-east-1' },
  environment: 'prod',
  domainName: 'go.yourdomain.com',
});
```

4

DynamoDB Table Creation

NoSQL storage for URL mappings

Table Design

Attribute	Type	Role	Example
shortCode	String	Partition Key (PK)	Xk3pQ
originalUrl	String	Attribute	https://aws.amazon.com/lambda/
createdAt	String	ISO timestamp	2024-06-01T12:00:00Z
ttl	Number	TTL (Unix epoch)	1780000000
clicks	Number	Atomic counter	1042
userId	String	GSI key (optional)	user#abc123

CDK: DynamoDB Construct

lib/constructs/database.ts

```
// lib/constructs/database.ts
import * as cdk from 'aws-cdk-lib';
import * as dynamodb from 'aws-cdk-lib/aws-dynamodb';
import { Construct } from 'constructs';

export class UrlDatabase extends Construct {
  public readonly table: dynamodb.Table;

  constructor(scope: Construct, id: string) {
    super(scope, id);

    this.table = new dynamodb.Table(this, 'UrlTable', {
      tableName: 'url-shortener',
      partitionKey: { name: 'shortCode', type: dynamodb.AttributeType.STRING },
      billingMode: dynamodb.BillingMode.PAY_PER_REQUEST, // Free Tier friendly
      encryption: dynamodb.TableEncryption.AWS_MANAGED,
      pointInTimeRecovery: true, // 35-day recovery window
      timeToLiveAttribute: 'ttl', // Automatic expiry of old URLs
      removalPolicy: cdk RemovalPolicy.RETAIN, // Never delete in prod
    });

    // GSI: look up all URLs created by a specific user
    this.table.addGlobalSecondaryIndex({
      indexName: 'userId-index',
      partitionKey: { name: 'userId', type: dynamodb.AttributeType.STRING },
    });
  }
}
```


lib/constructs/database.ts (cont.)

```
        sortKey:      { name: 'createdAt', type: dynamodb.AttributeType.STRING },
        projectionType: dynamodb.ProjectionType.ALL,
    });
}
```

✓ PAY_PER_REQUEST billing means \$0 cost when idle — ideal for dev/staging environments.

5

Lambda Functions

Shorten & Redirect handlers

Function 1: POST /shorten

Receives a JSON body with originalUrl, generates a 6-character nanoid short code, writes to DynamoDB with optional TTL, and returns the short URL.

lib/lambda/shorten.ts

```
// lib/lambda/shorten.ts
import { DynamoDBClient, PutItemCommand } from '@aws-sdk/client-dynamodb';
import { APIGatewayProxyEventV2, APIGatewayProxyResultV2 } from 'aws-lambda';
import { randomBytes } from 'crypto';

const ddb = new DynamoDBClient({ region: process.env.AWS_REGION });

function generateCode(length = 6): string {
  const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz23456789';
  return Array.from(randomBytes(length))
    .map(b => chars[b % chars.length])
    .join('');
}

export const handler = async (event: APIGatewayProxyEventV2): Promise<APIGatewayProxyResultV2> => {
  const body = JSON.parse(event.body || '{}');
  const { originalUrl, customCode, expiresInDays = 365 } = body;

  if (!originalUrl || !isValidUrl(originalUrl)) {
    return { statusCode: 400, body: JSON.stringify({ error: 'Invalid URL' }) };
  }

  const shortCode = customCode || generateCode();
  const ttl = Math.floor(Date.now() / 1000) + expiresInDays * 86400;

  await ddb.send(new PutItemCommand({
    TableName: process.env.TABLE_NAME!,
    Item: {
      shortCode: { S: shortCode },
      originalUrl: { S: originalUrl },
      createdAt: { S: new Date().toISOString() },
      ttl: { N: ttl.toString() },
      clicks: { N: '0' },
    },
    ConditionExpression: 'attribute_not_exists(shortCode)',
  }));

  return { statusCode: 200, body: JSON.stringify({ shortUrl: `https://example.com/${shortCode}` }) };
}
```

lib/lambda/shorten.ts (cont.)

```
    }));

    const shortUrl = `https://${process.env.DOMAIN}/${shortCode}`;
    return { statusCode: 201, body: JSON.stringify({ shortUrl, shortCode }) };
  };

const isValidUrl = (u: string) => { try { new URL(u); return true; } catch { return false; } };
```

Function 2: GET /{code} — Redirect Handler

lib/lambda/redirect.ts

```
// lib/lambda/redirect.ts
import { DynamoDBClient, GetItemCommand, UpdateItemCommand } from '@aws-sdk/client-dynamodb';
import { APIGatewayProxyEventV2, APIGatewayProxyResultV2 } from 'aws-lambda';

const ddb = new DynamoDBClient({ region: process.env.AWS_REGION });

export const handler = async (event: APIGatewayProxyEventV2): Promise<APIGatewayProxyResultV2> => {
  const shortCode = event.pathParameters?.code;
  if (!shortCode) return { statusCode: 400, body: 'Missing code' };

  const { Item } = await ddb.send(new GetItemCommand({
    TableName: process.env.TABLE_NAME!,
    Key: { shortCode: { S: shortCode } },
  }));

  if (!Item) return { statusCode: 404, body: JSON.stringify({ error: 'Not found' }) };

  // Async click counter increment (fire-and-forget)
  ddb.send(new UpdateItemCommand({
    TableName: process.env.TABLE_NAME!,
    Key: { shortCode: { S: shortCode } },
    UpdateExpression: 'ADD clicks :inc',
    ExpressionAttributeValues: { ':inc': { N: '1' } },
  }));

  return {
    statusCode: 301,
    headers: {
      Location: Item.originalUrl.S!,
      'Cache-Control': 'public, max-age=300', // 5-min CDN cache
    },
    body: '',
  };
}
```

lib/lambda/redirect.ts (cont.)

```
};  
};
```

CDK: Lambda Construct

CDK Lambda construct

```
// lib/url-shortener-stack.ts (Lambda section)  
const shortenFn = new lambda.Function(this, 'ShortenFn', {  
  runtime:      lambda.Runtime.NODEJS_20_X,  
  architecture: lambda.Architecture.ARM_64, // 20% cheaper  
  handler:      'shorten.handler',  
  code:         lambda.Code.fromAsset('lib/lambda'),  
  memorySize:   256,  
  timeout:      cdk.Duration.seconds(10),  
  environment: {  
    TABLE_NAME: urlTable.tableName,  
    DOMAIN:      props.domainName,  
  },  
  tracing: lambda.Tracing.ACTIVE, // X-Ray enabled  
});  
  
// Grant least-privilege IAM  
urlTable.grantWriteData(shortenFn);  
urlTable.grantReadWriteData(redirectFn);
```

6

API Gateway — HTTP API

Routing requests to Lambda

Why HTTP API v2 over REST API

Feature	HTTP API v2	REST API
Cost	\$1.00 / 1M req	\$3.50 / 1M req
Latency	Lower (direct Lambda)	Higher
JWT Authorizer	Native built-in	Lambda authorizer
WebSocket	Separate WS API	Separate WS API
Usage Plans	Not available	Available
Verdict	USE THIS ✓	Only if needed

CDK: API Gateway + Routes

lib/constructs/api.ts

```
// lib/constructs/api.ts
import * as apigwv2 from 'aws-cdk-lib/aws-apigatewayv2';
import { HttpLambdaIntegration } from 'aws-cdk-lib/aws-apigatewayv2-integrations';

const httpApi = new apigwv2.HttpApi(this, 'HttpApi', {
  apiName: 'url-shortener-api',
  description: 'Serverless URL Shortener HTTP API',
  corsPreflight: {
    allowOrigins: ['https://yourdomain.com'],
    allowMethods: [apigwv2.CorsHttpMethod.GET, apigwv2.CorsHttpMethod.POST],
    allowHeaders: ['Content-Type', 'Authorization'],
    maxAge: cdk.Duration.days(1),
  },
});

// POST /shorten – create short URL
httpApi.addRoutes({
  path: '/shorten',
  methods: [apigwv2.HttpMethod.POST],
  integration: new HttpLambdaIntegration('ShortenIntegration', shortenFn),
});

// GET /{code} – redirect
httpApi.addRoutes({
  path: '/{code}',
```

lib/constructs/api.ts (cont.)

```
    methods: [apigwv2.HttpMethod.GET],
    integration: new HttpLambdaIntegration('RedirectIntegration', redirectFn),
  });

// GET /stats/{code} - analytics
httpApi.addRoutes({
  path: '/stats/{code}',
  methods: [apigwv2.HttpMethod.GET],
  integration: new HttpLambdaIntegration('StatsIntegration', statsFn),
});

new cdk.CfnOutput(this, 'ApiUrl', { value: httpApi.url! });
```

✓ After deploy, test immediately: `curl -X POST $API_URL/shorten -d '{"originalUrl":"https://aws.amazon.com"}' -H 'Content-`

7

CloudFront & Custom Domain

Route 53 + ACM + CloudFront

Why CloudFront for a URL Shortener

- Edge caching: 301 redirects cached globally for 5 minutes — near-zero Lambda invocations for popular links
- TLS termination at the edge — sub-10ms HTTPS handshake vs 40ms+ from origin
- DDoS protection via AWS Shield Standard (included free)
- Custom domain: go.yourdomain.com instead of random CloudFront hostname
- Real-time logs to Kinesis for click analytics pipeline

Step-by-Step: Custom Domain Setup

1 Register domain or use existing

In Route 53 console → Registered Domains → Register domain, or use a domain registered elsewhere by creating a hosted zone.

2 Request ACM certificate

Navigate to ACM in us-east-1 (required for CloudFront). Request public certificate for go.yourdomain.com. Choose DNS validation.

3 Add CNAME to Route 53

ACM shows a CNAME record to add. In Route 53 → Hosted Zones → yourdomain.com → Create record → paste the CNAME. Wait 5-30 min for validation.

4 Create CloudFront distribution

Use CDK code below. Set the origin to the API Gateway URL, add the ACM cert ARN, and set the CNAME alias.

5 Add A record alias

In Route 53, create an A record for go.yourdomain.com pointing to the CloudFront distribution via Alias.

CDK: CloudFront Distribution

lib/constructs/cdn.ts

```
// lib/constructs/cdn.ts
import * as cloudfront from 'aws-cdk-lib/aws-cloudfront';
import * as origins from 'aws-cdk-lib/aws-cloudfront-origins';
import * as acm from 'aws-cdk-lib/aws-certificatemanager';
import * as route53 from 'aws-cdk-lib/aws-route53';
```

lib/constructs/cdn.ts (cont.)

```
import * as targets from 'aws-cdk-lib/aws-route53-targets';

const zone = route53.HostedZone.fromLookup(this, 'Zone', {
  domainName: 'yourdomain.com',
});

const certificate = new acm.Certificate(this, 'Certificate', {
  domainName: 'go.yourdomain.com',
  validation: acm.CertificateValidation.fromDns(zone),
});

const distribution = new cloudfront.Distribution(this, 'Distribution', {
  domainNames: ['go.yourdomain.com'],
  certificate,
  defaultBehavior: {
    origin: new origins.HttpOrigin(
      // Extract hostname from API GW URL
      `${httpApi.httpApiId}.execute-api.us-east-1.amazonaws.com`
    ),
    viewerProtocolPolicy: cloudfront.ViewerProtocolPolicy.REDIRECT_TO_HTTPS,
    cachePolicy: cloudfront.CachePolicy.CACHING_DISABLED, // API calls
    allowedMethods: cloudfront.AllowedMethods.ALLOW_ALL,
  },
  additionalBehaviors: {
    '/*': { // Redirects
      origin: new origins.HttpOrigin(...),
      cachePolicy: new cloudfront.CachePolicy(this, 'RedirectCache', {
        defaultTtl: cdk.Duration.minutes(5),
        maxTtl: cdk.Duration.minutes(5),
        headerBehavior: cloudfront.CacheHeaderBehavior.none(),
      }),
    },
  },
  webAclId: waf.webAclArn,
  enableLogging: true,
  logBucket: logBucket,
});
```


8

WAF Rules & Security Hardening

OWASP protection + rate limiting

WAF Rule Set

Rule	Type	Action
AWSManagedRulesCommonRuleSet	Core	OWASP Top 10 protection: SQLi, XSS, path traversal
AWSManagedRulesKnownBadInputsRuleSet	Known Bad	Blocks known malicious user-agents and payloads
Rate Limit — 1000 req/5min/IP	Custom	Throttles abusive clients; returns 429
Geo-Block (optional)	Custom	Block specific countries if compliance requires

CDK: WAF WebACL

WAF WebACL CDK

```
// lib/url-shortener-stack.ts (WAF section)
import * as wafv2 from 'aws-cdk-lib/aws-wafv2';

const webAcl = new wafv2.CfnWebACL(this, 'WebACL', {
  scope: 'CLOUDFRONT', // Must be us-east-1 for CloudFront
  defaultAction: { allow: {} },
  visibilityConfig: {
    cloudWatchMetricsEnabled: true,
    metricName: 'UrlShortenerWAF',
    sampledRequestsEnabled: true,
  },
  rules: [
    {
      name: 'AWSManagedRulesCommonRuleSet',
      priority: 1,
      overrideAction: { none: {} },
      statement: {
        managedRuleGroupStatement: {
          vendorName: 'AWS',
          name: 'AWSManagedRulesCommonRuleSet',
        },
      },
    },
  ],
  visibilityConfig: {
    cloudWatchMetricsEnabled: true,
    metricName: 'CommonRuleSet',
    sampledRequestsEnabled: false,
  },
});
```

WAF WebACL CDK (cont.)

```
    },  
  },  
  {  
    name: 'RateLimitRule',  
    priority: 2,  
    action: { block: {} },  
    statement: {  
      rateBasedStatement: {  
        limit: 1000,  
        aggregateKeyType: 'IP',  
      },  
    },  
    visibilityConfig: {  
      cloudWatchMetricsEnabled: true,  
      metricName: 'RateLimit',  
      sampledRequestsEnabled: true,  
    },  
  },  
],  
});
```

★ Enable WAF logging to S3 for forensics. Set log retention to 90 days minimum for compliance.

9

Monitoring: CloudWatch + X-Ray

Observability for production

CloudWatch Dashboard & Alarms

Create a CloudWatch dashboard with the following widgets and alarms to monitor your production service:

Alarm Name	Metric	Threshold	Action
HighErrorRate	Lambda Errors	> 1% over 5 min	SNS → Email
SlowRedirects	P99 Duration	> 500ms	SNS → PagerDuty
DDBThrottle	DDB Throttles	> 0	SNS → Email
4xxSpike	API GW 4xx	> 100/min	SNS → Slack
BillingAlert	EstCharges	> \$5/month	SNS → Email

CDK: Alarms + Dashboard

CloudWatch CDK

```
// Alarm: high error rate on Redirect Lambda
const errorAlarm = new cloudwatch.Alarm(this, 'RedirectErrors', {
  alarmName: 'url-shortener-redirect-errors',
  metric: redirectFn.metricErrors({
    period: cdk.Duration.minutes(5),
    statistic: 'Sum',
  }),
  threshold: 5,
  evaluationPeriods: 1,
  comparisonOperator: cloudwatch.ComparisonOperator.GREATER_THAN_THRESHOLD,
  treatMissingData: cloudwatch.TreatMissingData.NOT_BREACHING,
  alarmDescription: 'Redirect Lambda throwing errors – check X-Ray traces',
});
errorAlarm.addAlarmAction(new cwActions.SnsAction(alarmTopic));

// Dashboard
const dashboard = new cloudwatch.Dashboard(this, 'Dashboard', {
  dashboardName: 'UrlShortener-Production',
});
dashboard.addWidgets(
  new cloudwatch.GraphWidget({
    title: 'Invocations & Errors',
    left: [redirectFn.metricInvocations(), shortenFn.metricInvocations()],
    right: [redirectFn.metricErrors(), shortenFn.metricErrors()],
    width: 12,
  })
);
```

CloudWatch CDK (cont.)

```
    }),  
    new cloudwatch.GraphWidget({  
        title: 'P99 Duration (ms)',  
        left: [redirectFn.metricDuration({ statistic: 'p99' })],  
        width: 12,  
    }),  
);
```

Enable X-Ray Tracing

X-Ray traces the full request path: API Gateway → Lambda → DynamoDB. This gives you a service map and latency breakdown for every request.

Terminal — X-Ray CLI

```
# View recent X-Ray traces via CLI  
$ aws xray get-service-graph \  
    --start-time $(date -d '1 hour ago' +%s) \  
    --end-time $(date +%s) \  
    --region us-east-1  
  
# Get trace summaries  
$ aws xray get-trace-summaries \  
    --start-time $(date -d '30 minutes ago' +%s) \  
    --end-time $(date +%s) \  
    --filter-expression 'duration > 0.5' # Slow traces only
```

10

CI/CD Pipeline — GitHub Actions

Automated deploy on every push

Pipeline Architecture

1 Push to main

Developer pushes code to GitHub main branch

2 Tests & Lint

Jest unit tests + ESLint run in parallel (< 2 min)

3 CDK Synth & Diff

cdk synth produces CloudFormation; cdk diff shows changes

4 Deploy to Dev

Auto-deploy to UrlShortener-Dev stack on success

5 Integration Tests

curl-based smoke tests against Dev API endpoint

6 Manual Approval

GitHub Environments protection rule requires approval

7 Deploy to Prod

cdk deploy UrlShortener-Prod on approval

8 Notify Slack

Post deploy summary to #deployments channel

GitHub Actions Workflow

`.github/workflows/deploy.yml`

```
# .github/workflows/deploy.yml
name: Deploy URL Shortener
on:
```

.github/workflows/deploy.yml (cont.)

```
push:
  branches: [main]
pull_request:
  branches: [main]

permissions:
  id-token: write  # Required for OIDC
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm test
      - run: npm run lint

  deploy-dev:
    needs: test
    runs-on: ubuntu-latest
    environment: development
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::${{ secrets.AWS_ACCOUNT_ID }}:role/GitHubActionsDeploy
          aws-region: us-east-1
      - run: npm ci && npm run build
      - run: npx cdk deploy UrlShortener-Dev --require-approval never

  deploy-prod:
    needs: deploy-dev
    runs-on: ubuntu-latest
    environment: production  # Requires manual approval
    if: github.ref == 'refs/heads/main'
```

✓ Use OIDC (id-token: write) instead of static AWS_ACCESS_KEY_ID secrets for secure CI/CD.

11

Load Testing & Performance Tuning

Validate production readiness

Load Test with Artillery

Artillery Load Test

```
# Install Artillery
$ npm install -g artillery

# Create load test config: load-test.yml
config:
  target: 'https://go.yourdomain.com'
  phases:
    - duration: 60
      arrivalRate: 10
      name: Warm-up
    - duration: 120
      arrivalRate: 100
      name: Ramp-up
    - duration: 300
      arrivalRate: 500
      name: Peak load

  scenarios:
    - name: Shorten and Redirect
      flow:
        - post:
            url: '/shorten'
            json: { originalUrl: 'https://aws.amazon.com/lambda' }
            capture: { json: '$.shortCode', as: code }
        - get:
            url: '/{{ code }}'
            followRedirect: false
            expect: [{ statusCode: 301 }]

# Run the test
$ artillery run load-test.yml --output results.json
$ artillery report results.json
```

Expected Results & Tuning

Metric	Target	If Failing
P50 latency	< 20ms	Check CloudFront cache hit ratio

P99 latency	< 200ms	Enable Lambda Provisioned Concurrency
Error rate	< 0.1%	Check DynamoDB throttles in CloudWatch
Cold starts	< 1%	Use ARM64 + Node.js 20 for faster init

12

Cost Optimization & Clean-Up

Stay within Free Tier, clean up when done

Monthly Cost Breakdown (Free Tier)

Service	Free Tier	This Project	Cost
Lambda	1M req/month	~100k req	\$0
API Gateway	1M req/month (12mo)	~100k req	\$0
DynamoDB	25 RCU/WCU always	PAY_PER_REQUEST	~\$0
CloudFront	1TB/month always	< 1GB	\$0
Route 53	None	\$0.50/hosted zone	\$0.50
ACM	Free	1 cert	\$0
WAF	None	1 WebACL	\$5/mo
CloudWatch	5GB logs free	< 1GB	\$0
Total			~\$5.50

★ Disable WAF when not actively testing to avoid the \$5/month base charge during development.

Clean Up — Avoid Charges

Terminal — Resource Clean-up

```
# Destroy all CDK stacks and AWS resources
$ cdk destroy UrlShortener-Dev
$ cdk destroy UrlShortener-Prod

# Verify no lingering resources
$ aws dynamodb list-tables --region us-east-1
$ aws lambda list-functions --region us-east-1
$ aws cloudfront list-distributions

# Delete Route 53 hosted zone if created
$ aws route53 list-hosted-zones
$ aws route53 delete-hosted-zone --id /hostedzone/ZONEID

# Delete ACM certificates
$ aws acm list-certificates --region us-east-1
$ aws acm delete-certificate --certificate-arn arn:aws:acm:...

# Final billing check
```

Terminal — Resource Clean-up (cont.)

```
$ aws ce get-cost-and-usage \
  --time-period Start=2024-06-01,End=2024-06-30 \
  --granularity MONTHLY \
  --metrics BlendedCost
```

Congratulations! You have deployed a production-grade serverless application on AWS covering Lambda, API Gateway, DynamoDB, CloudFront, Route 53, ACM, WAF, CloudWatch, X-Ray, and CDK. This project demonstrates all key skills expected of an AWS Solutions Architect Associate and Professional. Add it to your portfolio with the GitHub repo link and architecture diagram.